

MongoDB

MongoDB es un sistema de base de datos NoSQL, de código abierto y orientado a documentos, diseñado para ofrecer alta escalabilidad, flexibilidad y rendimiento.

Almacena información en estructuras flexibles similares a JSON (llamadas BSON), lo que permite manejar datos no estructurados o semiestructurados sin esquemas rígidos.

Características y ventajas principales:

- **Modelo de Documentos:** En lugar de tablas y filas (como en SQL), utiliza colecciones y documentos, facilitando la representación de datos complejos.
- **Esquema Flexible:** Permite cambiar la estructura de los datos con el tiempo sin necesidad de redefinir tablas completas.
- **Escalabilidad Horizontal:** Diseñado para crecer distribuyendo datos a través de múltiples servidores (sharding).
- **Alto Rendimiento:** Ofrece indexación avanzada y consultas rápidas, ideal para grandes volúmenes de datos.
- **Uso versátil:** Ideal para aplicaciones modernas, gestión de contenidos, big data e Internet de las cosas (IoT).

En resumen, es una base de datos moderna orientada a la agilidad del desarrollador y la gestión de datos dinámicos.

Sharding:

El sharding (o fragmentación) es una técnica de arquitectura de bases de datos que divide un conjunto de datos muy grande en partes más pequeñas, manejables y rápidas, llamadas shards (fragmentos).

Cada fragmento se almacena en un servidor o nodo separado, lo que permite distribuir la carga de trabajo y mejorar el rendimiento del sistema.

En el contexto de las bases de datos NoSQL (Not Only SQL), el sharding es fundamental, ya que estas están diseñadas para escalar horizontalmente.

Conceptos Clave de Sharding en NoSQL

- **Escalabilidad Horizontal:** En lugar de comprar un servidor más potente (escalado vertical), el sharding permite añadir más servidores ("nodos") al clúster para manejar más datos y tráfico.
- **Particionado de Datos:** Los datos se distribuyen entre los nodos basándose en una "clave de partición" (shard key), lo que asegura que las consultas busquen solo en el nodo relevante, acelerando la respuesta.
- **Nativo en NoSQL:** A diferencia de las bases de datos SQL tradicionales, donde el sharding puede ser complejo, muchas bases de datos NoSQL (como MongoDB, Cassandra) tienen sharding nativo y automatizado.
- **Alta Disponibilidad:** Si un servidor (shard) falla, solo una parte de los datos se vuelve inaccesible, en lugar de toda la base de datos.

¿Por qué se usa Sharding en NoSQL?

- Manejo de Grandes Volúmenes: Ideal cuando el conjunto de datos supera la capacidad de un solo servidor, frecuentemente en el orden de los Terabytes (TB).
- Rendimiento: Al dividir la carga entre varios servidores, se reducen los cuellos de botella.
- Distribución Geográfica: Se pueden colocar fragmentos de datos cerca de los usuarios finales para reducir la latencia.

Tipos de Sharding (Estrategias)

- Basado en Rango (Range-based): Los datos se dividen según rangos de la clave (ej. usuarios A-M en Shard 1, N-Z en Shard 2).
- Basado en Hash: Se aplica una función hash a la clave para distribuir los datos de manera uniforme entre los nodos.- Desventaja: El sharding aumenta la complejidad del sistema y puede complicar la consistencia total de los datos entre todos los nodos.

[Documentacion oficial Mongo](#)

Redis

Redis (Remote Dictionary Server) es un almacén de estructuras de datos en memoria, de código abierto y tipo NoSQL clave-valor.

Es extremadamente rápido porque almacena datos en la RAM en lugar de discos, logrando latencias de microsegundos.

Se utiliza principalmente como caché, gestor de sesiones y corredor de mensajes.

Características y usos clave de Redis:

- Velocidad: Al trabajar en memoria, ofrece alto rendimiento y lectura/escritura veloz.
- Estructuras de Datos: Soporta múltiples tipos de datos como strings, hashes, listas, conjuntos (sets) y conjuntos ordenados.

Casos de uso:

- Caché: Reduce la carga en bases de datos tradicionales.
- Gestión de sesiones: Almacena sesiones de usuario, carritos de compras y tokens.
- Tiempo Real: Ideal para mensajería, chats y tablas de clasificación (leaderboards).

Persistencia:

A pesar de ser en memoria, ofrece opciones para guardar datos en disco.

Código abierto:

Es gratuito y cuenta con gran soporte de la comunidad.

Es una herramienta fundamental en arquitecturas modernas que requieren alta velocidad de respuesta.

ElasticSearch

Elasticsearch es un motor de búsqueda y analítica distribuido, de código abierto y basado en [Apache Lucene](#).

Permite almacenar, buscar y analizar grandes volúmenes de datos rápidamente en tiempo real, estructurados o no, indexándolos en formato JSON.

Es el componente central del "Elastic Stack" (ELK), ideal para búsquedas complejas, monitoreo de logs, seguridad (SIEM) y análisis.

Características clave:

- Búsqueda rápida y escalable: Utiliza índices invertidos para búsquedas de texto completo en milisegundos, escalando desde una máquina a cientos de servidores.
- Distribuido y resiliente: Los datos se dividen en fragmentos y se distribuyen entre nodos, garantizando alta disponibilidad.
- API RESTful: Interactúa con los datos mediante peticiones HTTP estándar.
- Versatilidad: Soporta búsqueda de texto, datos geográficos, métricas, y analítica avanzada.

Principales casos de uso:

- Búsqueda en sitios web/aplicaciones: Implementación de motores de búsqueda tipo "autocompletar" o de texto completo.- Monitoreo y logging: Análisis de logs de aplicaciones e infraestructura.
- Análisis de datos: Exploración de patrones y tendencias en tiempo real.

Elasticsearch se puede descargar e instalar localmente o utilizar a través de [Elastic Cloud](#) en la nube.

Apache Lucene

Apache Lucene es una biblioteca de software libre y de código abierto de alto rendimiento para búsqueda y recuperación de información, escrita en Java.

Permite a los desarrolladores integrar capacidades avanzadas de búsqueda de texto completo en aplicaciones, indexando grandes volúmenes de datos rápidamente.

Es el núcleo técnico de motores populares como Elasticsearch y Solr.

Características Principales:

- Biblioteca, no producto: Es un motor de búsqueda que se incorpora a las aplicaciones, no una aplicación independiente.
- Indexación rápida: Analiza texto y crea índices eficientes para búsquedas inmediatas.
- Búsqueda avanzada: Soporta búsquedas de texto completo, búsquedas booleanas, búsquedas con comodines y más.
- Escalabilidad: Diseñado para manejar grandes cantidades de datos.
- Multiplataforma: Aunque escrito en Java, existen puertos para otros lenguajes.

¿Para qué sirve?

Se utiliza comúnmente para implementar motores de búsqueda internos en sitios web, intranets, plataformas de correo electrónico, y en sistemas de Big Data.

Neo4j

Neo4j es la base de datos de grafos más popular y utilizada actualmente.

A diferencia de las bases de datos relacionales tradicionales (SQL) que usan tablas, Neo4j almacena datos en forma de nodos (entidades) y relaciones (conexiones).

Está diseñada para gestionar datos altamente conectados, permitiendo realizar consultas complejas y encontrar patrones en tiempo real.

Este [video](#) explica la diferencia entre el modelo relacional y el de grafos.

Características Principales

- Modelo de Grafos Nativo: Almacena los datos tal cual se conceptualizan (como nodos y relaciones).
- Lenguaje Cypher: Utiliza un lenguaje de consulta propio, declarativo y optimizado para grafos, similar a SQL pero diseñado para navegar relaciones.
- Alto Rendimiento: Es muy eficiente recorriendo grandes conjuntos de datos conectados.
- Transacciones ACID: Garantiza la seguridad y consistencia de los datos.
- Escalabilidad: Soporta la gestión de grandes volúmenes de datos y alta disponibilidad.

Este [video](#) introduce el lenguaje de consultas Cypher

¿Para qué se utiliza Neo4j?

Sus aplicaciones son ideales cuando las conexiones entre datos son tan importantes como los datos mismos, incluyendo:

- Detección de Fraudes: Identificar patrones inusuales en redes bancarias.
- Motores de Recomendación: Analizar relaciones para sugerir productos o amigos en tiempo real.
- Gestión de Redes y TI: Analizar la infraestructura de red.

- Redes Sociales: Mapear conexiones complejas.

Componentes Clave

- Nodos: Son las entidades (ej. "Persona", "Producto").
- Relaciones: Conectan los nodos (ej. "AMIGO_DE", "COMPRÓ").
- Propiedades: Información adicional almacenada dentro de nodos o relaciones.

Neo4j está implementada en Java y ofrece tanto una versión gratuita (Community) como una versión de pago (Enterprise).

Lenguaje Cypher

Una consulta básica en Neo4j utiliza el lenguaje Cypher para buscar patrones (MATCH) y devolver resultados (RETURN).

Por ejemplo, para encontrar a todas las personas llamadas "Juan", se usa:

```
MATCH (p:Persona {nombre: "Juan"}) RETURN p.
```

Cypher permite filtrar con WHERE, ordenar con ORDER BY y limitar resultados.

Ejemplos de Consultas Cypher

1. Consulta Elemental (Buscar nodos)

Devuelve todos los nodos de tipo Animal con la propiedad sexo igual a "Macho".

```
MATCH (n:Animal {sexo: "Macho"})  
RETURN n
```

2. Consulta Intermedia (Relaciones)

Encuentra todos los animales de los que se encarga un veterinario específico (ej. "Tadala").

```
MATCH (a:Animal {name: "Tadala"})<-[:SE_ENCARGA_DE]-(v:Empleado {puesto: "Veterinario"})  
MATCH (v)-[:SE_ENCARGA_DE]->(otrosAnimales:Animal)  
WHERE otrosAnimales.name <> "Tadala"  
RETURN otrosAnimales
```

3. Crear Nodos y Relaciones Crea un nodo de persona y una relación de "AMIGO" con otra persona existente.

```
MATCH (a:Persona {nombre: "Ana"}), (b:Persona {nombre: "Juan"})  
CREATE (a)-[:AMIGO]->(b)  
RETURN a, b
```

Componentes Clave de Cypher

- MATCH: Define el patrón de búsqueda (nodos y relaciones).(), []:
 - Los paréntesis () representan nodos, y
 - los corchetes [] representan relaciones.
 - -, <: Indican la dirección de la relación.
 - RETURN: Especifica los datos a mostrar.
 - WHERE: Filtra resultados por propiedades.
-
-
-

Solr

Apache Solr es un servidor de búsqueda de código abierto, rápido y de nivel empresarial, construido sobre la biblioteca Java de Apache Lucene.

Se utiliza para integrar motores de búsqueda avanzados, escalables y tolerantes a fallos en aplicaciones web, permitiendo búsquedas de texto completo en grandes volúmenes de datos en milisegundos.

Características clave de Solr:

- Basado en Lucene: Utiliza la potencia de Lucene para indexar y buscar documentos, permitiendo búsquedas precisas.
- Búsqueda avanzada: Ofrece búsqueda de texto completo, búsquedas facetadas (agrupación por categorías), resaltado de resultados y autocompletado.
- Gestión de documentos: Soporta la indexación de múltiples formatos como JSON, XML, CSV y PDF.
- Interfaz REST: Se interactúa con Solr mediante peticiones HTTP estándar (REST APIs), lo que facilita su uso desde cualquier lenguaje de programación.
- Escalabilidad (SolrCloud): Puede funcionar en un solo servidor o en modo distribuido (SolrCloud) para gestionar altos volúmenes de datos y garantizar la alta disponibilidad.

En resumen, Solr es la herramienta ideal cuando una aplicación necesita capacidades de búsqueda más complejas y rápidas que las que ofrece una base de datos relacional tradicional.

Consultas

Las consultas en Solr se realizan comúnmente a través de una API HTTP utilizando sintaxis Lucene, permitiendo búsquedas rápidas de texto completo, filtrado y facetado.

Los ejemplos clave incluyen:

- Búsquedas básicas (q=campo:valor)
- Búsquedas con comodines (*, ?), operadores booleanos (AND, OR, NOT)
- Búsquedas de rango ([min TO max]) para filtrar documentos

Búsquedas Básicas

- Buscar todo: `q=:` (Recupera todos los documentos).
- Búsqueda en campo específico: `q=nombre:developer` (Busca "developer" en el campo "nombre").
- Búsqueda de frase exacta: `q=titulo:"apache solr"` (Busca la frase exacta).

Operadores Booleanos

- AND: `q=nombre:solr AND tipo:motor` (Ambos términos deben coincidir).
- OR: `q=nombre:solr OR nombre:lucene` (Coincide al menos uno).
- NOT (Exclusión): `q=nombre:solr -tipo:beta` (Incluye solr, excluye beta).

Comodines y Búsqueda Difusa (Fuzzy)

- Asterisco (`*`): `q=nombre:desar` (Coincide con desarrollador, desarrollo, etc.).
- Signo de interrogación (`?`): `q=nombre:j?an` (Coincide con juan, joan).
- Fuzzy (Tolerancia a errores): `q=nombre:robert~1` (Busca nombres similares con 1 error, ej: Roberta, Robert).

Consultas de Rango y Rangos Numéricos

- Rango de fechas: `q=fecha_creacion:[2023-01-01T00:00:00Z TO NOW]`.
- Rango numérico: `q=precio:[10 TO 100]` (Busca precios entre 10 y 100).
- Edad específica: `q=age:30.5`.

Parámetros de Solr (Filtros y Facetas)

- Filtrar (`fq`): `q=laptop&fq=precio:[500 TO 1000]` (Busca "laptop", pero solo muestra las de \$500-\$1000).
- Faceta (Facet): `q=:&facet=true&facet.field=categoria` (Agrupa los resultados por campo "categoria").

Nota: Todas estas consultas se envían a través de la interfaz web de Solr (sección 'query') o mediante una URL HTTP (ej: <http://localhost:8983/solr/coleccion/select?q=...>).